freegrad: alternative backward rules and gradient transforms alongside PyTorch autograd

Luigi Troiano (ORCID)[1], Genny Tortora (ORCID)[1]

2025-08-24

[1]University of Salerno

1 Summary

Machine learning research often relies on backpropagation as implemented in PyTorch's **autograd**. While powerful, autograd enforces a strict symmetry between forward activations and their associated backward derivatives.
This coupling can limit experimentation with **alternative gradient rules**, which are relevant for studying optimization dynamics, robustness, and theoretical properties of neural networks.

We present **freegrad**, a lightweight extension to PyTorch that allows researchers to define, register, and apply **custom backward rules** while leaving the forward pass unchanged. freegrad provides a registry of gradient transforms, a context manager for selective application, and wrappers for activation layers, enabling experiments on gradient clipping, stochastic perturbations (*gradient jamming*), or forward/backward decoupling.

2 Statement of need

Recent theoretical and empirical work has explicitly questioned the necessity of forward-backward symmetry in neural network training. Troiano et al. demonstrated that gradient direction, primarily determined by linear neuron connections, is the dominant factor in learning effectiveness, while exact gradient magnitudes derived from activation functions are largely redundant (Troiano et al. 2025). They introduced constant, rectangular, triangular, and noisy gradients as alternatives, showing that neural networks—including CNNs and BNNs—can be trained effectively under these conditions. This motivates the need for tools that allow researchers to experiment with **custom backward rules** without patching PyTorch autograd.

freegrad directly addresses this need by providing a modular framework to replicate, extend, and generalize such experiments in a reproducible way.

3 Functionality

freegrad includes:

- **Registry of gradient rules**: built-in rules include `rectangular`, `triangular`, `clip_norm`, `noise`, and jamming variants (`full_jam`, `positive_jam`, `rectangular_jam`).
- **Context manager**: `with xg.use(rule, params, scope):` temporarily applies a rule during backpropagation.
- **Activation wrappers**: drop-in replacements for ReLU, Tanh, Sigmoid, etc., with custom backward functions controlled by the context.
- **Hooks**: optional parameter hooks for rule application at the parameter gradient level.

- **Functional API**: experimental support for custom `jvp` and `vjp`.

freegrad is tested with PyTorch >=2.0, documented with MkDocs, and distributed under the MIT license.

4 Acknowledgements

We thank the open-source PyTorch community for providing the foundation on which freegrad builds. This project was inspired by research on gradient dynamics and experimental work on *gradient jamming*.

References

[] Troiano, Luigi, Francesco Gissi, Vincenzo Benedetto, and Genny Tortora. 2025. "Breaking the Conventional Forward-Backward Tie in Neural Networks: Activation Functions." *Neurocomputing* 654: 131178. `https://doi.org/10.1016/j.neucom.2025.131178`.